

Development Plan Pentachess

Version 0

Team 6
Brandon Bosman
Mian Rafay
Karl Sader
Saad Tariq
Emaan Khan

Table 1: Revision History

Date	Developer(s)	Change
October 25, 2024	All	Initial Draft

An online multiplayer chess game featuring the Pentachess variant.

1. Team Meeting Plan

- We will hold weekly meetings on Mondays at 2:30 pm. Every other week, our meeting will be a sprint review, where we discuss the results of the previous two-week sprint and plan goals for the next sprint. These will be held either in person or over video call. The meetings in between these will be stand-ups, where we monitor each other's progress and discuss issues and blockers. These can be held by text or by video call on Discord.
- We will also have monthly meetings to discuss how the functional and non-functional requirements are being met. We will include our supervisor Dr. Rapoport in these when we have significant questions for him to answer.
- There will also need to be meetings for the 'subteams', e.g., for the frontend or backend teams. The organization of these meetings can be informal and left up to the individuals within each subgroup.

2. Team Communication Plan

- Discord will be our primary communication method. We will use it for moment-to-moment informal communication, such as planning meetings and discussing smaller topics that don't need to be discussed in a meeting. We will create multiple channels on our Discord server to discuss different features in parallel.

- GitHub will also be used for communication in some contexts. We will use the project board to keep track of each big-picture goal and the sub-goals within each goal. We will also use the issue tracker and pull requests to discuss more focused topics, i.e. individual features.
- Jira will be used to plan and monitor our sprints. We will also use it to organize our backlog and to track issues.
- Google Docs will be used for creating the main documents required alongside the project, including the project description, SRS document, development plan, and VnV plan. We will share the document between ourselves and use the built-in collaboration feature to work on documents concurrently.

3. Team Member Roles

- **Brandon Bosman:** Program Manager/Frontend/Backend
- **Mian Rafay:** Backend
- **Karl Sader:** Backend
- **Saad Tariq:** Frontend/Backend
- **Emaan Khan:** Frontend

Table 2: Requirements each team member will be working on

Team Member	Requirement Responsibility	Detailed Requirement
Brandon Bosman	Game Setup and Logic (with Mian and Karl)	• Online matchmaking
	User Account Management (with Karl)	• Registering for an account/login credentials • Option for password recovery
	Game Setup (with Saad)	• Ability to start/join/forfeit a local game
	Timer	• Updating timer countdown+state+end game on 0 time
Mian Rafay	Game Setup and Logic (with Karl and Brandon)	• Algorithms for valid moves, checks, check/stalemate • API for front/backend communication
	Timer	• Save timer state after each turn
	Game Session	• Expiry of session after periods of inactivity
Karl Sader	Game Setup and Logic (with Mian and Brandon)	• Algorithms for valid moves, checks, check/stalemate • Online matchmaking • API for front/backend communication
	User Account Management (with Brandon)	• Ability to log in to an existing account (authentication)
Saad Tariq	Game Setup (with Brandon)	• Ability to start/join/forfeit an online game
	Interactions (with	• Displaying board and all pieces (with colour indicators)

	Emaan)	• End game on detection of check/stalemate+winner message • Responsive UI that adjusts to screen size
Emaan Khan	Interactions (with Saad)	• Ability to drag/drop pieces from your side • Highlight where a selected piece can go • Updating board state when a piece is moved
	Game Session	• Expiry of game session upon logout/forfeit/inactivity

Our coordinator/program manager is Brandon Bosman. He will ensure the availability of all team members for our bi-weekly online group meetings. He will also be overlooking where our resources need to be allocated. Suppose a certain group/person needs assistance with a heavily weighted task. In that case, he will decide how to allocate said task more efficiently (usually by allocating his time to that task).

4. Workflow Plan

GitHub WorkFlow:

- **Git Branching Strategy:** The GitFlow branching strategy will enable parallel development amongst developers to ensure features and fixes are being pushed efficiently. There will be two primary branches that developers will be interacting, one being the stable branch of the project, and the other is where features are first merged for testing and integration:
 - **Main Branch:** This branch contains the most stable and fully tested code, it is also the branch that will be deployed.
 - **Development Branch:** This branch is where features and fixes will be integrated/pushed into for testing purposes. Once all code within the dev branch is ready for deployment, it will be merged with the main branch.
 - **Feature Branch:** This branch will contain any specific feature to be implemented within the project, it will follow the naming convention of **feat-xxx-xxx** (i.e. feat-create-login). Once the feature is developed, it will be merged into the dev branch for further testing and analysis.
- **Git Commit Conventions:** All Git commits must follow the same strict conventions to ensure that when other developers are going through files within the codebase in GitHub, they can view which commit made the last changes and should have a good idea of what the changes are, based on the commit message. Commit messages should follow the following format, “**prefix: name of commit**” (i.e. “feat: add register route”). Here is a list of all commit message prefixes that will be used during this project:
 - **fix:** For commits that fix a bug.
 - **feat:** Adding a new feature to the project.
 - **refactor:** Restructuring/rewriting code that does not change functionality.
 - **style:** Changes that do not affect the meaning of the code (white-space, formatting, missing semicolons, etc).
 - **test:** Adding new test cases or correcting existing ones.
 - **docs:** Adding in documentation or any other project document.
 - **chore:** Commits that don't change source code or tests.
- **Pull Request (PR) Conventions:** All pull requests must follow the same detailed naming and description conventions to ensure that code can be tracked correctly and that bad code or any errors can be found swiftly.

- PR titles must follow git commit conventions, followed by a more detailed but concise title (e.g. “Feat: Add Login Functionality to Home Screen”).
- The description of the PR must be detailed with valid screenshots and explanations for the changes within the codebase.
- PRs must have links to any GitHub issue if there is one, with a “#” followed by the issue number.
- All code reviews must take place within the PR comment section.
- The PR must be reviewed and approved by at least one other team member.
- The PR must be squashed and merged into the dev branch.
- All commits within the PR must be squashed into one commit.
- **Issue management:** Custom templates will be used for feature and documentation requests and bugs:
 - Labels such as bugs, documentation, help wanted, and questions should be used within each issue.
 - Sprints or any other project phases will be assigned to issues to ensure GitHub is the main source of development information for developers.

Agile Methodology: We will follow an agile methodology to develop software to ensure that we meet all requirements and are closely engaged with the product owner in the development process.

- **Scrum Master:** A project manager who ensures that all work is done smoothly, and any potential blockage is prevented. This role will be fulfilled by Brandon Bosman.
- **Product Owner:** The primary stakeholder who is actively involved in the project, conveying their vision of the product. This role is fulfilled by Dr. Paul Rapoport.
- **Development Team:** Individuals developing the product. This role is fulfilled by Emaan Khan, Saad Tariq, Karl Sader, Main Rafay, and Brandon Bosman.
- **Agile/Scrum Framework:** We will use Jira for planning sprints, organizing backlogs, and tracking any bugs or issues alongside GitHub.
 - **Sprints:** A sprint is a segment of time in which all planned technical and non-technical work must be completed. The project workload must be divided into sprints that will be two weeks long.
 - **Sprint Planning:** This will take place at the start of each sprint, where all tasks for each team member are allocated. Note that all work must strictly be finished before the end of each sprint.
 - **Stand-ups:** Stand-ups will take place bi-weekly to ensure team members are making progress in their tasks and to ensure developers are not being blocked. This is also the time to discuss any issues or questions regarding the project. These meetings can take place via Discord video call or text.
 - **Sprint Review:** This will take place at the beginning of each sprint before planning the next sprint. This is to ensure that tasks and goals are being met and to discuss any improvements for upcoming sprints.

Data Storage: Within this project, we will only store user and session data in a PostgreSQL relational database.

Tools to Meet Functional Requirements: We will use Next.js/React.js for frontend development, Node.js for backend development, and Socket.io for real-time connection.

Tools to Measure Metrics: We will use Vitest, Playwright and Supertest to test load times and functionality within the frontend and backend testing including database connections. We will also be conducting manual tests for performance and functionality.

5. Proof of Concept Demonstration Plan

The main risk for the success of our project is underestimating the complexity of the project. From a glance, it may seem like a straightforward project, but going into the details, there are a lot of minor requirements we must consider. In addition, as we begin implementation, one functional requirement may end up breaking down into multiple, causing an expansion of our project tasks. As long as proper planning is taken before the implementation of this project, everything should run smoothly, making our project a success.

To overcome the risk of complexity underestimation, in our proof-of-concept demonstration, we will demonstrate the UI of our 90-tile board and all its pieces. The pieces will be draggable to another tile on the board. We will also aim to implement game logic, so pieces can only move to tiles they are allowed to. Presenting this in our proof-of-concept demonstration provides a reasonable amount of work and preparation that won't lead to a large expansion of our project's tasks.

6. Technology

Languages:

- **Frontend:** TypeScript, HTML, CSS
- **Backend:** TypeScript

Linters: ESLint

Testing:

- **Unit testing and code coverage:** Vitest
- **Frontend E2E Testing:** Playwright
- **API Request Testing:** Supertest

Continuous Integration (CI):

We use a basic GitHub Actions workflow to lint, build, and test the code whenever a pull request is made to the development branch. This ensures that code quality is maintained throughout the development process.

Frameworks and Libraries:

- **Frontend:** Next.js, React.js, Tailwind CS
- **Backend:** Express.js (web server), Prisma (ORM),
- **Both:** Socket.io (WebSockets), Zod (runtime type validation)

Tools and Environments:

- **Backend Runtime:** Node.js
- **Database:** PostgreSQL
- **API Documentation:** Swagger
- **Code Formatting:** Prettier
- **IDE:** Visual Studio Code (VSCode)

7. Coding Standard

Code should conform to Google's TypeScript style guide, found at:

<https://google.github.io/styleguide/tsguide.html> **Guide** and follow general clean code concepts found at: <https://github.com/ryanmcdermott/clean-code-javascript>.

Naming:

Variable naming should be descriptive and adhere to the guidelines found at: <https://github.com/kettanaito/naming-cheatsheet> and follow these casing convents:

- **PascalCase:** Classes, interfaces, types and React.js component names.
- **camelCase:** Variables, functions, object keys, React.js hooks.
- **UPPER_SNAKE_CASE:** Constants (e.g. API_KEY).

Design Principles:

Code should adhere to common design principles:

- **SOLID Principles:** Follow the SOLID design principles for structuring code, especially in the more complex parts of the backend.
- **Dry (Don't Repeat Yourself):** Avoid code duplication by abstracting shared functionality into utility functions or common components.

Design Patterns:

Incorporate common design patterns where applicable to solve recurring problems:

- **Singleton:** share a single instance throughout the application (e.g. Database client).
- **Observer Pattern:** Use observables to notify subscribers when an event occurs (e.g. event listeners, Websockets).
- **Module Pattern:** Split up code into smaller and reusable pieces.

For more design patterns and examples, refer to: <https://www.patterns.dev>.

The overall goal is to ensure the code is readable, maintainable and easily extensible.

8. Project Scheduling

With the November 21st project demo fast approaching, our team plans to build an acceptable prototype that will succeed in providing a general overview of our project. Our team has strategically set goals with realistic deadlines to ensure project development aligns with important dates provided by the project supervisor.

Goal 1: Setup

1. Create a project repository on GitHub ensuring all developers have access and can contribute changes.
2. Create an organized layout of basic files and the code structure.
3. Setup Next.js for the frontend and Express.js for the backend.
4. Connect the frontend with the backend for reliable communication between the two by defining API endpoints.
5. Setup the relevant routes for pages on Next.js.
6. Setup a PostgreSQL database connection and implement a chessboard model that allows us to store game states.

Goal 2: Implement Board and Pieces

1. Decide on the theme for the board and the general layout of the web application.
2. Implement the board along with the pieces on the frontend.
3. Ensure the pieces have a draggable property that allows them to move seamlessly across the board through player interaction by mouse or touch.
4. Implement code for movements of all various pieces.
5. Highlights all cells to which the selected piece can legally move.

Goal 3: Implement Game Logic

1. Keep track of the game status every turn. Check for checks, checkmates, stalemates, and captured pieces.
2. Implement offline local play with the option for a custom timer to keep track of each player's remaining time.
3. Display all captured pieces in a side panel viewable by both players.
4. Write code to determine the end of game sessions due to checkmate, stalemate, no time remaining, or forfeit.

Goal 4: Enable Online Play

1. Implement an account system for users to register and login to existing accounts.
2. Setup sessions using Express.js.
3. Implement a lobby system for players to join and create private or public games.
4. Establish WebSocket communication to enable real-time play.
5. Ensure both players can move pieces turn by turn with real-time updates, and update the database with corresponding game states.
6. Extend the code to determine the end of game sessions to account for loss of connection from either side resulting in an automatic forfeit and upon logging out.

Figure 1: Gantt Chart

Work:  No Classes: 

Week Task	Week 1 (Oct 21)	Week 2 (Oct 28)	Week 3 (Nov 4)	Week 4 (Nov 11)	Week 5 (Nov 18)	Week 6 (Nov 25)	Week 7 (Dec 2)
Goal 1							
Goal 2							
Goal 3							
Goal 4							
Final Testing							

Week Task	Week 8 (Dec 9)	Week 9 (Dec 16)	Week 10 (Dec 23)	Week 11 (Dec 30)	Week 12 (Jan 6)	Week 13 (Jan 13)	Week 14 (Jan 20)
Goal 1							
Goal 2							
Goal 3							
Goal 4							
Final Testing							

Week Task	Week 15 (Jan 27)	Week 16 (Feb 3)	Week 17 (Feb 10)	Week 18 (Feb 17)	Week 19 (Feb 24)	Week 20 (Mar 3)	Week 21 (Mar 10)	
Goal 1								
Goal 2								
Goal 3								
Goal 4								
Final Testing								

Week Task	Week 22 (Mar 17)	Week 23 (Mar 24)	Week 24 (Mar 31)
Goal 1			
Goal 2			
Goal 3			
Goal 4			
Final Testing			